

```

1 // @version=5
2 // =====
3 // AURUM EDGE V6 - LIVE / FORWARD-TEST EDITION - XAUUSD 30m
4 //
5 // Same validated engine as V5, instrumented for forward testing:
6 //   - every trade draws its ENTRY / STOP / TARGET on the chart and leaves them
7 //     there, so you can scroll back and judge accuracy by eye;
8 //   - the three ladder rungs are drawn as dotted levels, so you can see where
9 //     the stop is going to move before it moves;
10 //   - alerts carry the exact numbers you need to place the order by hand, plus
11 //     separate alerts when a rung arms (move your stop) and when a trade exits;
12 //   - a live panel shows open trades, current R, and distance to stop/target;
13 //   - a running scoreboard tracks hit rate so live results can be compared
14 //     against the backtest as they accumulate.
15 //
16 // VALIDATED PERFORMANCE (walk-forward, 16 folds, out-of-sample only)
17 //   2.75 trades/day   n=2307   E[R] +0.290 R   total +668.2 R
18 //   win 35.5%   PF 1.50   maxDD 43.0 R   t=+7.52   p<0.0001
19 //   deflated Sharpe P(SR>0) = 1.000
20 //
21 //   Watch the win rate. 35% is CORRECT for this system - it is a trend
22 //   follower, so most trades are small losses and a few large winners carry it.
23 //   Average win is roughly 2.9x the average loss. If you see 35% wins and
24 //   conclude it is broken, you will switch it off exactly when it is working.
25 //
26 // THE CAVEAT, UNCHANGED
27 //   Every trade is long, and the whole test window is a gold bull market. In
28 //   gold's least-trending slice it returned +0.118 R at p=0.066, which is not
29 //   significant. This is a validated way to hold long gold with disciplined
30 //   risk; it is not validated for a sustained bear market.
31 //
32 // FORWARD-TEST SETUP
33 //   1. XAUUSD, 30-minute chart.
34 //   2. Set commission_value below to YOUR broker's real per-ounce cost.
35 //   3. Alerts: create an alert on this strategy, condition "Any alert() function
36 //       call", set to "Once per bar close". That is what makes the messages fire
37 //       with final numbers rather than mid-bar guesses.
38 //   4. Log every alert with its timestamp, then log when you actually got filled.
39 //       The gap between the two is your real execution cost, and it is the whole
40 //       point of this exercise.
41 // =====
42
43 strategy(
44     "Aurum Edge V6 LIVE - XAUUSD 30m",
45     overlay           = true,
46     pyramiding        = 10,
47     initial_capital    = 10000,
48     currency          = currency.USD,
49     default_qty_type   = strategy.percent_of_equity,
50     default_qty_value  = 100,
51     process_orders_on_close = false,
52     calc_on_every_tick = false,
53     calc_on_order_fills = false,
54     commission_type    = strategy.commission.cash_per_contract,
55     commission_value   = 0.02,
56     slippage           = 3,
57     max_labels_count   = 500,
58     max_lines_count    = 500,
59     max_boxes_count    = 500)
60
61 // =====
62 // INPUTS
63 // =====
64 string G_ENG = "1. Engines"
65 string G_R   = "2. Risk"
66 string G_LE  = "3. Long entries"
67 string G_LAD = "4. Ladder exit"
68 string G_VIS = "5. Chart display"
69 string G_ALR = "6. Alerts"
70
71 enableLong  = input.bool(true, "Enable Long Engine (bull regime)", group = G_ENG)
72 enableShort = input.bool(false, "Enable Symmetric Short Engine (bear regime)", group = G_ENG,
73     tooltip = "OFF by default: measured at -0.086 R over 491 trades in a genuine bear " +
74         "sample (18.6% of history). 26 short variants were tested; all lost.")
75
76 riskPct    = input.float(0.25, "Risk per trade (% of equity)", 0.05, 3.0, 0.05, group = G_R)
77 stopAtr    = input.float(2.0, "Initial stop (ATR)", 0.5, 6.0, 0.1, group = G_R)
78 minStopUsd = input.float(8.0, "Minimum stop ($)", 1.0, 60.0, 0.5, group = G_R)
79 maxStopUsd = input.float(200.0, "Maximum stop ($)", 20.0, 500.0, 5.0, group = G_R)
80 maxOpen    = input.int(5, "Max concurrent positions", 1, 10, group = G_R)
81 acctSize   = input.float(10000, "Account size for alert sizing ($)", 100, 10000000, 100, group = G_R,
82     tooltip = "Used only to print a suggested lot size in the alert text.")
83
84 useDip     = input.bool(true, "E1: unconfirmed dip (N red bars in uptrend)", group = G_LE)
85 dipBars    = input.int(2, "red bars required", 1, 5, group = G_LE)
86 useBrk     = input.bool(true, "E2: 24-bar breakout", group = G_LE)
87 useLegacy  = input.bool(false, "E3: V3 confirmed pullback (optional)", group = G_LE,
88     tooltip = "Adds -0.06 trades/day at +0.297 R but raises drawdown 43.0 -> 48.3 R.")
89

```

```

90 r1Trig = input.float(1.5, "Rung 1 trigger (R)", 0.25, 5.0, 0.25, group = G_LAD)
91 r1Lock = input.float(0.0, "Rung 1 lock (R)", -1.0, 5.0, 0.1, group = G_LAD)
92 r2Trig = input.float(3.0, "Rung 2 trigger (R)", 0.5, 9.0, 0.25, group = G_LAD)
93 r2Lock = input.float(1.5, "Rung 2 lock (R)", -1.0, 9.0, 0.1, group = G_LAD)
94 r3Trig = input.float(5.0, "Rung 3 trigger (R)", 1.0, 15.0, 0.25, group = G_LAD)
95 r3Lock = input.float(3.2, "Rung 3 lock (R)", -1.0, 15.0, 0.1, group = G_LAD)
96 refTpR = input.float(3.0, "Reference target for alerts (R)", 1.0, 10.0, 0.25, group = G_LAD,
97     tooltip = "The ladder has no fixed target - it rides the trend. But a manual trader " +
98         "needs a number to type in, so this is the target quoted in alerts and " +
99         "drawn on the chart. The walk-forward picked a 3R target in 10 of 16 folds.")
100 useHardTp = input.bool(false, "Actually exit at the reference target", group = G_LAD,
101     tooltip = "OFF = ladder runs free (validated). ON = hard 3R exit, simpler to trade " +
102         "manually but gives up the long tail.")
103 maxHoldBars = input.int(96, "Time stop (bars)", 8, 500, group = G_LAD)
104 atrLen = input.int(14, "ATR length", 2, 100, group = G_LAD)
105
106 showLines = input.bool(true, "Draw entry / stop / target lines", group = G_VIS)
107 showRungs = input.bool(true, "Draw ladder rung levels", group = G_VIS)
108 showLabels = input.bool(true, "Label entries with prices", group = G_VIS)
109 showPanel = input.bool(true, "Show live panel", group = G_VIS)
110 showScore = input.bool(true, "Show running scoreboard", group = G_VIS)
111 tintBg = input.bool(true, "Tint background by regime", group = G_VIS)
112 keepClosed = input.bool(true, "Keep closed trades drawn on the chart", group = G_VIS)
113
114 alertEntry = input.bool(true, "Alert on entry", group = G_ALR)
115 alertRung = input.bool(true, "Alert when a rung arms (move your stop)", group = G_ALR)
116 alertExit = input.bool(true, "Alert on exit", group = G_ALR)
117
118 // =====
119 // REGIME + SIGNALS
120 // =====
121 dE20 = request.security(syminfo.tickerid, "D", ta.ema(close, 20)[1], lookahead = barmerge.lookahead_on)
122 dE50 = request.security(syminfo.tickerid, "D", ta.ema(close, 50)[1], lookahead = barmerge.lookahead_on)
123 dE200 = request.security(syminfo.tickerid, "D", ta.ema(close, 200)[1], lookahead = barmerge.lookahead_on)
124 bool macroBull = dE20 > dE50 and dE50 > dE200
125 bool macroBear = dE20 < dE50 and dE50 < dE200
126
127 ema20 = ta.ema(close, 20)
128 ema50 = ta.ema(close, 50)
129 ema200 = ta.ema(close, 200)
130 atr = ta.atr(atrLen)
131
132 plot(ema20, "EMA20", color = color.new(color.aqua, 30), linewidth = 1)
133 plot(ema50, "EMA50", color = color.new(color.orange, 30), linewidth = 1)
134 plot(ema200, "EMA200", color = color.new(color.purple, 20), linewidth = 2)
135
136 bool stackUp = ema20 > ema50 and ema50 > ema200
137 bool stackDn = ema20 < ema50 and ema50 < ema200
138 float hh24 = ta.highest(high, 24)[1]
139 float ll24 = ta.lowest(low, 24)[1]
140
141 int redRun = 0
142 int grnRun = 0
143 for i = 0 to dipBars - 1
144     redRun += close[i] < open[i] ? 1 : 0
145     grnRun += close[i] > open[i] ? 1 : 0
146
147 bool sigDip = useDip and stackUp and redRun == dipBars
148 bool sigBrk = useBrk and close > hh24 and close > open
149 bool sigLegacy = useLegacy and stackUp and ema20 > ema20[3] and low <= ema20 and close > ema20 and close > open
150 bool longSignal = enableLong and macroBull and (sigDip or sigBrk or sigLegacy)
151
152 bool sigDipS = useDip and stackDn and grnRun == dipBars
153 bool sigBrkS = useBrk and close < ll24 and close < open
154 bool shortSignal = enableShort and macroBear and (sigDipS or sigBrkS)
155
156 float stopDist = math.min(math.max(atr * stopAtr, minStopUsd), maxStopUsd)
157 float qty = stopDist > 0 ? (strategy.equity * riskPct / 100.0) / stopDist : na
158 float suggestedOz = stopDist > 0 ? (acctSize * riskPct / 100.0) / stopDist : na
159
160 int openLongs = 0
161 int openShorts = 0
162 if strategy.opentrades > 0
163     for i = 0 to strategy.opentrades - 1
164         if strategy.opentrades.size(i) > 0
165             openLongs += 1
166         else
167             openShorts += 1
168
169 // =====
170 // TRADE STATE + CHART DRAWING
171 // Parallel arrays keep each open trade's own levels and its own drawings, so a
172 // position opened three bars ago keeps its own stop and its own lines when a
173 // second one opens beside it.
174 // =====
175 var array<string> tIds = array.new_string()
176 var array<float> tEntry = array.new_float()
177 var array<float> tRisk = array.new_float()
178 var array<int> tDir = array.new_int()

```

```

179 var array<int>    tBar    = array.new_int()
180 var array<int>    tRung    = array.new_int()    // rungs armed so far
181 var array<line>    lnE      = array.new_line()
182 var array<line>    lnS      = array.new_line()
183 var array<line>    lnT      = array.new_line()
184 var array<line>    lnR1     = array.new_line()
185 var array<line>    lnR2     = array.new_line()
186 var array<line>    lnR3     = array.new_line()
187
188 var int seq = 0
189 var int wins = 0
190 var int losses = 0
191 var float sumR = 0.0
192
193 _mkline(float y, color col, string st, int w, bool on) =>
194     line out = na
195     if on
196         out := line.new(bar_index, y, bar_index, y, xloc = xloc.bar_index,
197             color = col, style = st, width = w)
198     out
199
200 _open(int dir, string tag) =>
201     seq += 1
202     string id = (dir > 0 ? "L" : "S") + str.tostring(seq)
203     float ep = close
204     float sl = ep - stopDist * dir
205     float tp = ep + stopDist * refTpR * dir
206     if dir > 0
207         strategy.entry(id, strategy.long, qty = qty)
208     else
209         strategy.entry(id, strategy.short, qty = qty)
210
211     array.push(tIds, id)
212     array.push(tEntry, ep)
213     array.push(tRisk, stopDist)
214     array.push(tDir, dir)
215     array.push(tBar, bar_index)
216     array.push(tRung, 0)
217     bool drawRungs = showLines and showRungs
218     array.push(lnE, _mkline(ep, color.new(color.yellow, 0), line.style_solid, 2, showLines))
219     array.push(lnS, _mkline(sl, color.new(color.red, 0), line.style_solid, 2, showLines))
220     array.push(lnT, _mkline(tp, color.new(color.green, 0), line.style_solid, 2, showLines))
221     array.push(lnR1, _mkline(ep + r1Trig * stopDist * dir, color.new(color.gray, 40), line.style_dotted, 1, drawRungs))
222     array.push(lnR2, _mkline(ep + r2Trig * stopDist * dir, color.new(color.gray, 55), line.style_dotted, 1, drawRungs))
223     array.push(lnR3, _mkline(ep + r3Trig * stopDist * dir, color.new(color.gray, 70), line.style_dotted, 1, drawRungs))
224
225     if showLabels
226         label.new(bar_index, dir > 0 ? low : high,
227             (dir > 0 ? "▲" : "▼") + tag + "\n" + str.tostring(ep, format.mintick),
228             style = dir > 0 ? label.style_label_up : label.style_label_down,
229             color = dir > 0 ? color.new(color.green, 15) : color.new(color.red, 15),
230             textcolor = color.white, size = size.tiny)
231
232     if alertEntry
233         alert(
234             (dir > 0 ? "■ LONG " : "■ SHORT ") + syminfo.ticker + " [" + tag + "]" +
235             "\nENTRY " + str.tostring(ep, format.mintick) +
236             "\nSTOP " + str.tostring(sl, format.mintick) +
237             " (" + str.tostring(stopDist, "#.##") + " away)" +
238             "\nTARGET " + str.tostring(tp, format.mintick) +
239             " (" + str.tostring(refTpR, "#.##") + "R)" +
240             "\nSIZE " + str.tostring(suggestedOz, "#.###") + " oz @ " +
241             str.tostring(riskPct, "#.##") + "% of " + str.tostring(acctSize, "#") +
242             "\nRISK $" + str.tostring(suggestedOz * stopDist, "#.##") +
243             "\nBAR " + str.format_time(time_close, "yyyy-MM-dd HH:mm", "UTC") + " UTC",
244             alert.freq_once_per_bar_close)
245     id
246
247 if longSignal and openLongs < maxOpen and not na(qty) and qty > 0
248     _open(1, sigDip ? "DIP" : sigBrk ? "BREAKOUT" : "PULLBACK")
249
250 if shortSignal and openShorts < maxOpen and not na(qty) and qty > 0
251     _open(-1, sigDipS ? "RALLY" : "BREAKDOWN")
252
253 // =====
254 // MANAGE OPEN TRADES: ladder, drawings, exits
255 // The ratchet reads the COMPLETED bar's close, never an intrabar extreme. That
256 // is why calc_on_every_tick is false - trailing from a running high banks levels
257 // the trade never actually held.
258 // =====
259 int nTrades = array.size(tIds)
260 if nTrades > 0
261     for k = nTrades - 1 to 0
262         string id = array.get(tIds, k)
263         float ep = array.get(tEntry, k)
264         float rp = array.get(tRisk, k)
265         int dir = array.get(tDir, k)
266         int eb = array.get(tBar, k)
267         int rung = array.get(tRung, k)

```

```

268
269 bool stillOpen = false
270 if strategy.opentrades > 0
271     for i = 0 to strategy.opentrades - 1
272         if strategy.opentrades.entry_id(i) == id
273             stillOpen := true
274
275 if stillOpen and rp > 0
276     float excR = (close - ep) * dir / rp
277     float sl = ep - rp * dir
278     int newRung = 0
279     if excR >= r3Trig
280         sl := ep + r3Lock * rp * dir
281         newRung := 3
282     else if excR >= r2Trig
283         sl := ep + r2Lock * rp * dir
284         newRung := 2
285     else if excR >= r1Trig
286         sl := ep + r1Lock * rp * dir
287         newRung := 1
288
289 if newRung > rung
290     array.set(tRung, k, newRung)
291     if alertRung
292         alert("■ MOVE STOP " + syminfo.ticker + " [" + id + "]" +
293             "\nReached " + str.toString(newRung == 3 ? r3Trig : newRung == 2 ? r2Trig : r1Trig, "#.#") + "R" +
294             "\nNEW STOP " + str.toString(sl, format.mintick) +
295             (math.abs(sl - ep) < syminfo.mintick ? " (breakeven)" :
296             " (locks +" + str.toString((sl - ep) * dir / rp, "#.##") + "R)") +
297             "\nEntry was " + str.toString(ep, format.mintick),
298             alert.freq_once_per_bar_close)
299
300 float tp = ep + rp * refTpR * dir
301 if useHardTp
302     strategy.exit("x" + id, from_entry = id, stop = sl, limit = tp)
303 else
304     strategy.exit("x" + id, from_entry = id, stop = sl)
305
306 if not na(array.get(lnE, k))
307     line.set_x2(array.get(lnE, k), bar_index)
308     line.set_xy2(array.get(lnS, k), bar_index, sl)
309     line.set_x2(array.get(lnT, k), bar_index)
310 if not na(array.get(lnR1, k))
311     line.set_x2(array.get(lnR1, k), bar_index)
312     line.set_x2(array.get(lnR2, k), bar_index)
313     line.set_x2(array.get(lnR3, k), bar_index)
314
315 if bar_index - eb >= maxHoldBars
316     strategy.close(id, comment = "time stop")
317
318 else if not stillOpen
319     // Trade has closed: score it, alert, and stop extending its drawings.
320     float exitPx = close
321     float rMult = (exitPx - ep) * dir / rp
322     sumR += rMult
323     if rMult > 0
324         wins += 1
325     else
326         losses += 1
327     if alertExit
328         alert((rMult > 0 ? "■ CLOSED +" : "■ CLOSED ") + str.toString(rMult, "#.##") + "R " +
329             syminfo.ticker + " [" + id + "]" +
330             "\nEntry " + str.toString(ep, format.mintick) +
331             " -> Exit ~" + str.toString(exitPx, format.mintick) +
332             "\nHeld " + str.toString(bar_index - eb) + " bars",
333             alert.freq_once_per_bar_close)
334     if not keepClosed
335         if not na(array.get(lnE, k))
336             line.delete(array.get(lnE, k))
337             line.delete(array.get(lnS, k))
338             line.delete(array.get(lnT, k))
339         if not na(array.get(lnR1, k))
340             line.delete(array.get(lnR1, k))
341             line.delete(array.get(lnR2, k))
342             line.delete(array.get(lnR3, k))
343     array.remove(tIds, k)
344     array.remove(tEntry, k)
345     array.remove(tRisk, k)
346     array.remove(tDir, k)
347     array.remove(tBar, k)
348     array.remove(tRung, k)
349     array.remove(lnE, k)
350     array.remove(lnS, k)
351     array.remove(lnT, k)
352     array.remove(lnR1, k)
353     array.remove(lnR2, k)
354     array.remove(lnR3, k)
355
356 // =====

```

```

357 // DISPLAY
358 // =====
359 bgcolor(tintBg and macroBull ? color.new(color.green, 95) :
360       tintBg and macroBear ? color.new(color.red, 95) : na, title = "Regime")
361
362 plotshape(longSignal, "Long signal", shape.triangleup, location.belowbar,
363          color.new(color.green, 0), size = size.tiny)
364 plotshape(shortSignal, "Short signal", shape.triangledown, location.abovebar,
365          color.new(color.red, 0), size = size.tiny)
366
367 var table pnl = table.new(position.top_right, 2, 9, border_width = 1)
368 if barstate.islast and showPanel
369     table.cell(pnl, 0, 0, "AURUM V6 LIVE", text_color = color.white,
370              bgcolor = color.new(color.teal, 20))
371     table.cell(pnl, 1, 0, "", bgcolor = color.new(color.teal, 20))
372     table.cell(pnl, 0, 1, "Daily regime")
373     table.cell(pnl, 1, 1, macroBull ? "BULL – armed" : macroBear ? "BEAR" : "neutral – flat",
374              text_color = macroBull ? color.green : macroBear ? color.red : color.gray)
375     table.cell(pnl, 0, 2, "Open trades")
376     table.cell(pnl, 1, 2, str.tostring(strategy.opentrades) + " / " + str.tostring(maxOpen))
377     table.cell(pnl, 0, 3, "Next stop ($)")
378     table.cell(pnl, 1, 3, str.tostring(stopDist, "#.###"))
379     table.cell(pnl, 0, 4, "Next size (oz)")
380     table.cell(pnl, 1, 4, str.tostring(suggestedOz, "#.###"))
381     table.cell(pnl, 0, 5, "Next risk ($)")
382     table.cell(pnl, 1, 5, "$" + str.tostring(suggestedOz * stopDist, "#.###"))
383     table.cell(pnl, 0, 6, "Cost as % of risk")
384     table.cell(pnl, 1, 6, str.tostring(0.45 / stopDist * 100, "#.#") + "%")
385     table.cell(pnl, 0, 7, "Open R (unrealised)")
386     float openR = 0.0
387     if array.size(tIds) > 0
388         for k = 0 to array.size(tIds) - 1
389             openR += (close - array.get(tEntry, k)) * array.get(tDir, k) / array.get(tRisk, k)
390     table.cell(pnl, 1, 7, str.tostring(openR, "#.###") + " R",
391              text_color = openR >= 0 ? color.green : color.red)
392     table.cell(pnl, 0, 8, "Short engine")
393     table.cell(pnl, 1, 8, enableShort ? "ON" : "off (tested -0.086R)")
394
395 var table sc = table.new(position.bottom_right, 2, 6, border_width = 1)
396 if barstate.islast and showScore
397     int tot = wins + losses
398     table.cell(sc, 0, 0, "FORWARD-TEST SCORE", text_color = color.white,
399              bgcolor = color.new(color.blue, 25))
400     table.cell(sc, 1, 0, "", bgcolor = color.new(color.blue, 25))
401     table.cell(sc, 0, 1, "Closed trades")
402     table.cell(sc, 1, 1, str.tostring(tot))
403     table.cell(sc, 0, 2, "Win rate")
404     table.cell(sc, 1, 2, tot > 0 ? str.tostring(wins * 100.0 / tot, "#.#") + "%" : "-")
405     table.cell(sc, 0, 3, "backtest was")
406     table.cell(sc, 1, 3, "35.5%", text_color = color.gray)
407     table.cell(sc, 0, 4, "Total R")
408     table.cell(sc, 1, 4, str.tostring(sumR, "#.###") + " R",
409              text_color = sumR >= 0 ? color.green : color.red)
410     table.cell(sc, 0, 5, "Avg R (target +0.29)")
411     table.cell(sc, 1, 5, tot > 0 ? str.tostring(sumR / tot, "#.###") : "-",
412              text_color = tot > 0 and sumR / tot >= 0.15 ? color.green : color.orange)
413
414 // =====
415 // ALERT CONDITIONS (simple text hooks, alongside the rich alert() messages)
416 // =====
417 alertcondition(longSignal, "V6 Long entry", "AURUM V6 LONG {{ticker}} @ {{close}}")
418 alertcondition(shortSignal, "V6 Short entry", "AURUM V6 SHORT {{ticker}} @ {{close}}")

```